

LONGER: 工业推荐中的长序列建模扩展

LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders

基于用户上传论文的中文逐段翻译整理版

摘要

在工业推荐系统中，对超长用户行为序列进行建模，对于同时捕捉长期偏好与短期偏好至关重要。现有方案通常依赖两阶段召回或间接建模范式，这会带来上游与下游不一致以及计算效率低下的问题。本文提出 LONGER，即一种面向 GPU 高效推荐系统的长序列优化 Transformer (Long-sequence Optimized traNsformer for GPU-Efficient Recommenders)。LONGER 包含：(i) 一种全局 token 机制，用于在长上下文中稳定注意力；(ii) 一种带有轻量级 InnerTransformer 和混合注意力策略的 token merge 模块，用于降低二次复杂度；以及 (iii) 一系列工程优化，包括混合精度训练与激活重计算、KV cache 在线服务，以及一个全同步的模型训练与服务框架，以统一基于 GPU 的稠密参数与稀疏参数更新。LONGER 在字节跳动的广告与电商业务中，无论在线下指标还是在线 A/B 测试中，都持续优于强基线模型，从而验证了其稳定有效性以及工业级扩展规律。目前，LONGER 已在字节跳动数十个真实且有影响力的业务场景中完成验证并全面部署，服务数十亿用户。

CCS 概念

信息系统 → 推荐系统。

关键词

超长序列建模；工业推荐系统；扩展规律。

1 引言

在推荐系统中，超长用户历史行为序列能够较为完整地刻画用户的长期偏好与短期偏好 [7,8]。尽管早期序列建模架构已经在学术界和工业界得到了广泛研究和广泛应用，但其应用场景仍主要局限于短序列情形（序列长度通常为 $10^2 \sim 10^3$ ）。

对长序列（长度大于 10^3 ）进行完整建模，能够显著提升推荐准确率与多样性，并有助于缓解“信息茧房”现象。然而，受限于计算开销，当前工业界在长序列建模中的既有主流做法主要采用以下策略：

- **两阶段检索**。从原始超长序列中选取与当前候选物品最相关的 top- k 个物品（通常 k 约为 10^2 ），然后再进行端到端的短序列建模。代表性工作包括 SIM [18] 和 TWIN [3,21]。
- **预训练用户嵌入** [9,13,31]。在工业界，常见做法是先在源模型中对整个超长序列进行预训练，得到压缩后的用户嵌入（UE），再将其迁移到下游推荐模型中。借助高性能先进 GPU，这类方法通常支持长度可达 10^3 的序列预训练以及多层 Transformer 建模。
- **记忆增强模型**。多通道用户兴趣记忆网络（MIMN） [17] 基于神经图灵机与记忆归纳单元结构实现用户序列记忆；大记忆网络（LMN） [14] 提出了一种基于乘积量化分解的轻量结构；而记忆增强推荐模型（MARM） [15] 则提出了一种以存储换计算的范式，将计算量较大的模块中间结果缓存起来。

尽管上述策略显著提升了计算效率，但由于上游与下游不一致，或者只能间接感知原始超长序列，它们不可避免地牺牲了原始完整序列信息。因此，这些方法本质上仍然只是通向端到端长序列建模过程中的中间阶段。

近年来，以 GPT [19] 为代表的大语言模型快速发展，建立了所谓的扩展规律 (scaling laws)，即随着模型规模、数据规模和算力的提升，模型性能会遵循经验规律持续提高。近期，这些扩展规律也开始指导推荐系统的创新。例如，HSTU [25] 采用由残差连接串联的同构自注意力层堆叠来建模长序列，其性能优于标准 Transformer 架构；Wukong [26] 则构建了基于堆叠因子分解机与线性压缩模块的交互架构，并在推荐场景中验证了扩展规律。

与此同时，随着计算基础设施的快速进步（例如 GPU FLOPs、显存能力，以及大规模工程计算平台与框架），工业级推荐系统中端到端超长序列建模范式的落地已经成为可能。因此，推进超长序列的端到端建模，并持续扩展序列长度、改进长序列建模架构，已经成为下一代序列建模框架发展的关键任务。

基于此，本文提出面向 GPU 高效推荐系统的长序列优化 Transformer，简称 LONGER。在该框架中，我们将输入序列组织为全局 token 与原始序列两部分，并基于此设计了一种基于 inner-transformer 的 token merge 方法，以有效降低计算预算。此外，考虑到用户超长序列中往往存在较多噪声，我们进一步采用高效的混合注意力策略，在保持模型性能的同时提升计算效率。最后，为了使 LONGER 能在十亿级用户规模的工业场景中真正落地，我们还提出了一系列工程优化，包括基于混合精度与激活重计算的全同步训练-服务框架，以及 KV cache 服务策略。总体而言，本文的主要贡献概括如下：

- 我们提出了 LONGER，一种面向 GPU 高效推荐系统的长序列优化 Transformer 结构。该方法从工业 GPU 高效性的角度出发，对 Transformer 结构进行优化，并首次在工业场景中将用户序列建模长度以端到端方式扩展到 10,000。
- LONGER 通过 token merge 与混合注意力策略显著提升了计算效率，在几乎无性能损失的前提下约减少了 50% FLOPs。同时，我们还设计了一个经过充分优化的工业级训练与服务框架，以进一步提升 GPU 计算效率并支持线上部署。
- 我们进行了全面实验来验证方法有效性。在线下，基于十亿级工业数据集开展实验；在线上，在抖音两个具有重要影响力的业务场景中进行了 A/B 测试。当前，LONGER 已在字节跳动数十个业务场景中广泛应用，影响数十亿用户。

2 相关工作

2.1 传统短序列建模

迄今为止，工业推荐系统主要遵循“序列建模 + 特征交互”的联合建模范式 [22,24]。在该框架中，序列建模一直是刻画用户偏好的核心组成部分。在大量研究中，一个重要的里程碑工作是 DIN [30]，其后又出现了 DIEN [29]、CAN [28] 等方法。此外，多域建模 [2,4]、多兴趣建模 [1,11] 以及序列去噪方法 [5,20] 也分别从不同角度被广泛用于用户偏好建模。需要指出的是，这些结构设计大多仍面向短序列建模，而长序列建模方法则是在之后才逐渐受到更多关注。

2.2 长序列建模

正如引言中所讨论的，长序列建模方法大致可以分为两阶段检索、预训练用户嵌入以及记忆增强模型三类。总体而言，基于检索的方法和基于预训练的方法都属于两阶段策略，而记忆增强模型通常还需要较长训练周期，才能在记忆槽中积累足够的命中率。近年来，也有一些工作开始尝试直接建模长序列 [25,27,32]。然而，在大规模工业推荐系统中，面向 GPU 高效性的长序列建模仍然缺乏充分研究。

3 方法

3.1 问题定义

设 U 和 I 分别表示用户集合与物品集合。对于一个用户 $u \in U$ ，给定其原始行为序列 $S_u = [i_1^{(u)}, \dots, i_L^{(u)}]$ ，其中 $i_t^{(u)} \in I$ ；给定用户基础特征 u_d ，包括用户画像、上下文特征与交叉特征；再给定一个目标物品 $v \in I$ ，推荐任务旨在预测点击或转化概率：

$$P(y = 1 \mid S_u, u_d, v) \in [0, 1] \quad (1)$$

其中 $y \in \{0, 1\}$ 表示用户 u 是否会与目标物品 v 发生交互。模型通过历史交互数据集 $D = \{(S_u, u_d, v, y)\}$ 学习上述映射，并优化二元交叉熵损失：

$$\mathcal{L} = -\frac{1}{|D|} \sum_{(S_u, u_d, v, y) \in D} [y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \quad (2)$$

其中 $\hat{y} = f_\theta(S_u, v)$ 表示推荐模型预测得到的概率。

3.2 整体框架

我们提出的框架旨在解决推荐系统中长而复杂的用户行为序列建模问题，同时在工业规模下保持训练与推理效率。图 1 展示了我们提出模型 LONGER 的整体结构。该框架融合了输入构造、token merge、混合注意力机制以及训练与服务优化，从而支持高效、可扩展的长序列建模。

首先，我们在模型输入结构中引入了 **Global Tokens**，其作为聚合锚点表示（例如目标物品表示、用户 ID 嵌入等），用于促进全局信息融合并稳定注意力分布。接着，我们采用 **Token Merge** 对长行为序列进行压缩，从而在保留关键局部模式的同时降低计算复杂度。为了进一步保留组内依赖关系，我们引入了 **InnerTrans**，即在合并后的 token 分组内部应用的轻量级 inner transformer。核心模型结构采用了一种混合注意力设计，结合了 **交叉因果注意力**（用于突出序列中重要部分）与 **堆叠自因果注意力层**（用于捕捉序列中的高阶依赖）。

为了保证方法具备扩展性并能够真正部署，我们还融入了若干工程级系统优化。该框架支持同步的训练与服务，并在超大规模 GPU 集群上统一存储和更新稠密参数与稀疏参数。我们进一步利用 **混合精度训练** 与 **重计算** 提升显存与算力效率，从而减少激活显存占用，并支持自定义数值精度。最后，在推理阶段，我们部署了 **KV Cache Serving** 策略，通过缓存用户序列表征并在多个候选打分过程中复用它们，以显著减少重复计算。

这些组件共同构成了一个完整系统，使其能够以较高表达能力和较高效率支持长序列建模，并且能够方便地部署到大规模真实推荐场景中。

3.3 全局 Token

我们引入 **Global Tokens** 作为附加在输入序列上的辅助表示，用于促进全局信息提取与锚定。这些 token 可以包括目标物品表示 token、可学习的 CLS token、UID 嵌入，以及高阶压缩的用户-物品交互特征。按照设计，这些全局 token 拥有完整的注意力感受野，因此能够从整个序列中聚合上下文信号，同时也能够反向影响其他所有序列 token。

这种结构增强主要服务于两个目的。其一，全局 token 作为集中式信息锚点，能够增强用户历史、上下文属性与候选物品之间的特征交互。其二，它们能够稳定长序列中的注意力动态，尤其是在稀疏注意力配置下更为明显。正如 StreamLLM [23] 所展示的那样，引入少量全局 token 能够缓解“attention sink”效应，即在更深层注意力网络中，注意力会不成比例地集中到靠前的 token 上。这些全局 token 作为锚点，有助于保持注意力多样性，并保留长程依赖建模能力。

3.4 Token Merge

设序列长度为 L ，嵌入维度为 d 。对于长行为序列（通常 $L \geq 2000$ ），使用标准 Transformer 进行建模会带来极高的计算代价，因为其注意力复杂度为二次型 $\mathcal{O}(L^2d)$ ，尤其是在 $L \gg d$ 的工业推荐场景中更为明显（例如典型设置中 $L = 2000, d = 32$ ）。传统的解决方式如序列截断，会导致长程依赖信息丢失。为此，我们提出 **Token Merge** 策略，将相邻 token 分组并压缩成更短序列，在模型效率与表征保真度之间取得折中。该策略可将序列长度缩短为原来的 $1/K$ ，相当于进行了一种“空间压缩”。分组后的 token 表示可以通过简单拼接得到，也可以进一步通过轻量级 InnerTrans 块增强组内交互。这种设计在保留局部语义的同时，使模型能够在更短序列上进行全局建模，并且在效率与表达能力之间提供灵活权衡。

对于一个标准结构的 Transformer 编码层，其 FLOPs 与参数量可以表示为 [16]:

$$\text{FLOPs}_{\text{vanilla trans}} = 24Ld^2 + 4L^2d \quad (3)$$

$$\text{Params}_{\text{vanilla trans}} = 12d^2 + 13d \quad (4)$$

计算复杂度。 在 token merge 前后，注意力复杂度之比为：

$$\frac{\text{FLOPs}_{\text{Merge Token}}}{\text{FLOPs}_{\text{vanilla}}} = \frac{24Ld^2/K + 4L^2d/K}{24Ld^2 + 4L^2d} = \frac{6d/K + L/K}{6d + L} \quad (5)$$

对于典型设置 $L = 2048, d = 32$:

- 标准 Transformer: FLOPs 约为 587 M;
- 当 $K = 4$ 时: FLOPs 约为 336 M, 减少约 42.8%。

参数扩展。 Token merging 通过缩短序列长度降低计算复杂度，同时会增加参数量 Θ_{merge} ，从而在提高效率的同时也提升模型表达能力，进而有助于整体性能提升：

$$\Theta_{\text{merge}} = 12K^2d^2 + 13Kd \quad (6)$$

InnerTrans。 若将多个相邻 token 合并为一个 token，仅通过简单拼接组内 token，可能会导致 token 之间交互不足，进而造成细粒度信息损失。为解决这一问题，我们引入 **InnerTrans**，即在每个 token 组内部使用一个 transformer 来建模局部交互。这能够保证每个分组内部的关系得到充分捕获，从而避免直接拼接所带来的信息损失。由于该模块中的维度与序列长度都很小，因此其计算开销在实践中非常有限。具体形式为：

$$M_i = \text{TransformerBlock}([e_i^1, \dots, e_i^K]) \quad (7)$$

其中， M_i 表示第 i 个分组的表示， e_i^k 表示第 i 组中第 k 个 item 的嵌入。

3.5 LONGER 模型结构

在模型架构中，我们采用一种混合注意力机制，同时结合交叉注意力层与自注意力层，以高效处理输入序列。

3.5.1 输入生成

模型输入由两部分构成：全局 token 与序列 token。全局 token 表示上下文信息（如第 3.3 节所述的目标物品特征与用户标识），并与序列 token 拼接形成整体输入。

为了更好地刻画用户行为序列的时间动态，我们还向序列 token 中加入了额外的位置侧信息。具体而言，我们引入两类位置编码：(1) 绝对时间差特征，用于量化每次用户交互与目标物品之间的时间距离，该特征作为侧信息与 item embedding 进行拼接；(2) 可学习的绝对位置嵌入，用于编码每个 token 在序列中的位置，并直接加到 item embedding 上。

加入位置编码后，所得 token 会通过一个多层感知机 (MLP) 生成输入表示： $R \in \mathbb{R}^{(m+L) \times d} = [G \in \mathbb{R}^{m \times d}; H \in \mathbb{R}^{L \times d}]$ ，其中 G 与 H 分别表示全局 token 表示与序列 token 表示。随后，查询矩阵 O 由 m 个全局 token $G \in \mathbb{R}^{m \times d}$ 与从完整序列 token H 中根据预定义采样策略选出的 k 个序列 token $H_S \in \mathbb{R}^{k \times d}$ 拼接构成。类似的查询压缩思想在其他研究领域中也有所体现，例如 Perceiver [10] 与 Q-Former [12] 都采用了可学习 token 进行压缩。在实验中，我们系统比较了多种策略，包括取最近的 k 个 token、均匀采样 token，以及初始化 k 个可学习 token，最终发现“最近 k 个”效果最佳。这种混合注意力设计还受到如下观察的启发：模型性能相对于序列 token 数量存在明显边际效应，仅采样完整序列的 40% 就能保留超过 95% 的性能收益，同时大约减少 50% FLOPs (见第 4 节)。最终，组合查询表示为：

$$O = [G; H_S] \quad (8)$$

这种混合设计将注意力同时聚焦于关键局部行为与全局上下文信号，从而使模型能够高效捕捉具体序列依赖以及更广泛的上下文信息。

3.5.2 交叉因果注意力（第一层）

在第一层注意力中，我们对前一步构造的查询矩阵 O 与输入 token $R \in \mathbb{R}^{(m+L) \times d}$ 应用交叉因果注意力。其形式为：

$$Q = OW_Q, \quad K = RW_K, \quad V = RW_V \quad (9)$$

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d}} + M \right) V \quad (10)$$

其中， W_Q, W_K, W_V 分别表示形状为 $\mathbb{R}^{d \times d}$ 的 query、key 与 value 投影矩阵，而掩码矩阵 M 定义为：

$$M_{i,j} = \begin{cases} 0, & \text{若 } j \geq i, \{i, j\} \in [1, m+L] \\ -\infty, & \text{否则} \end{cases} \quad (11)$$

这种因果掩码设计一方面保持了序列项之间的时间相关性，另一方面确保序列对候选物品不可见，从而使 KV Cache Serving 机制（见第 3.6.3 节）成为可能。完成注意力计算后，结果还会进一步通过前馈网络 (FFN) 处理。

3.5.3 自因果注意力（后续层）

在交叉因果注意力层之后，后续层由若干个自因果注意力块构成。这些层重点学习采样 token 序列内部的关系，使模型能够捕捉行为序列 token 之间的依赖结构与模式。每个自因果注意力层后都接一个 FFN，用于对注意力学习到的信息进行进一步处理。其形式与前述类似：

$$\text{SelfAttention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} + M \right) V \quad (12)$$

这里的 query、key 与 value 均由上一层输出分别经过线性投影 W_Q, W_K, W_V 得到。

3.5.4 堆叠与压缩

自因果注意力层会堆叠 N 次，以逐步细化输入序列的表示。经过这些层之后，模型得到一个压缩输出，该输出即为注意力机制的最终结果，随后被用于下游预测任务。

$$\text{CrossAttn}(O, R) \xrightarrow{\text{压缩长序列}} \text{SelfAttn}(\cdot) \times N \xrightarrow{\text{建模高阶交互}} \text{最终表示} \quad (13)$$

通过在第一层使用交叉注意力、在后续层使用自注意力，我们的模型能够在高效处理长序列的同时，同时利用全局上下文信息与序列内部依赖关系。

3.6 训练与部署优化

3.6.1 训练框架

我们的训练框架是一个专为大规模稀疏模型设计的全同步系统，旨在充分利用现代高性能 GPU 的能力。该框架基于“硬件-软件协同设计”理念，目标是在分布式训练中最大化计算吞吐与显存效率。训练流程从批式或流式数据摄取开始，随后通过 **Fountain** 模块进行预处理，处理后的训练数据会被分发到多个 GPU runner，在这些 runner 上同步更新稠密参数与稀疏参数。这种统一设计能够有效支持跨设备、跨节点扩展，为生产环境中训练大参数模型提供坚实基础。

该框架的一个关键特征是其统一的参数存储与训练架构。稠密参数与稀疏参数都存储在 GPU 机器上，并进行同步更新，从而不再需要外部参数服务器 (Parameter Server) 组件。为了更好适应推荐系统中常见的特征分布模式，框架对稀疏 embedding 采用分层存储体系，以高效支持大规模 embedding 表。在该设计中，高频特征存储在高带宽 GPU 显存 (HBM) 中，中频特征驻留在 CPU 主存 (MEM) 中，而低频特征则被卸载到本地固态硬盘 (SSD) 上。这种分层存储布局针对推荐数据的访问特性进行了优化，在时延、吞吐与容量之间取得了实用折中。其核心创新在于将计算与参数存储都完全共置于 GPU 机器上，从而降低通信开销与内存传输延迟，最终带来更高训练吞吐、更低参数陈旧性以及更稳定的收敛过程。

3.6.2 混合精度训练与重计算

为了缓解训练过程中的 GPU 显存压力，我们同时采用了重计算策略与混合精度训练。对于梯度计算，我们使用反向模式自动微分。该方式相比前向模式更高效，但需要保存前向传播中的所有中间激活，这些激活可能成为显存瓶颈。为此，我们支持在模型定义层面声明可重计算模块，使某些激活在前向阶段被丢弃，并在反向阶段重新计算，以“增加计算换取节省显存”。由于原生 TensorFlow 未正式支持重计算，我们利用 `custom_gradient` 机制实现该功能，从而通过代码级标注实现细粒度控制。

此外，为了降低稠密模型扩展带来的算力开销，我们采用基于 BF16/FP16 的混合精度训练。用户可以在模型层面配置数值精度，对关键模块使用更高精度，对其他部分使用更低精度。该方法在生产负载中展现出显著收益，平均可带来约 +18% 吞吐、-16% 训练时间、-18% 显存占用，而在稠密层中显存下降幅度最高可达 -28%。

3.6.3 KV Cache Serving

为了在对多个候选进行打分时提升推理效率，受 M-FALCON [25] 启发，我们引入一种 KV 缓存机制，将用户行为 token 与候选相关全局 token 之间的注意力计算解耦。由于同一用户在多个候选打分时其行为序列保持不变，因此其内部表示可以只计算一次并被复用。

具体而言，我们将注意力输入划分为两部分：(1) 用户序列 token；(2) 与候选物品相关的全局 token。用户序列的 key 与 value 投影会被预先计算并缓存。对于每个候选，只需计算其全局 token 与缓存用户序列之间的注意力。因此，推理过程分为两步：

1. 预先计算并缓存用户序列的 key-value 张量；
2. 计算每个候选的全局 token 与缓存用户序列之间的注意力。

如原文图 3 所示，这种优化避免了重复计算，并显著降低了服务时延。在实际场景中，它能够显著提高在线服务效率，将吞吐退化从最高约 -40% 降低到仅 -6.8%。

4 实验

4.1 实验设置

我们在抖音广告系统中的转化率（CVR）预测任务上评估所提模型，该任务属于一个真实的大规模工业广告推荐场景。数据集由 2024 年 10 月 16 日至 2025 年 2 月 23 日之间采集的一部分线上用户交互日志构成，共覆盖连续 130 天、包含 52 亿样本。每个样本包括用户人口属性特征（如用户 ID、性别）、超长用户行为序列以及一个候选广告 item。用户行为序列包含多种交互类型，包括页面浏览、点击与转化；而物品侧特征则包含广告内容、展示上下文与相关元数据。我们采用时间一致的数据划分策略：前 123 天用于训练，后 7 天用于线下评测。这种设置符合真实部署实践，并有效避免了模型开发过程中的未来数据泄漏。

为了进行比较，我们将所提模型与若干强基线进行对比，这些基线依据其对短程或长程用户行为的建模能力进行分类。短序列方法包括 TWIN [3] 与 DIN (Recent50)，它们仅依赖 50 次交互。长序列方法包括 SumPooling、DIN [30]、HSTU [25] 与 Transformer [6]，它们处理更长的行为历史，但在工业环境中通常会受到可扩展性与效率问题的限制。所有模型均采用相同的数据预处理流程与超参数调优策略，实验运行于一个由 48 张 A100 GPU 构成的集群上。

4.2 整体性能

4.2.1 与现有方法的比较

我们在线下评测集上使用推荐系统二分类任务中的两个标准指标评估模型表现：AUC（ROC 曲线下面积）与 LogLoss。表 1 总结了多个基线模型与所提模型的结果。根据表中结果，我们的方法优于所有基线，取得了 0.85290 的 AUC 与 0.47103 的 LogLoss。与基础模型相比，AUC 相对提升 1.57%；与最强竞争模型 Transformer 相比，AUC 也进一步提升了 0.21%。需要指出的是，在工业场景的在线 A/B 测试中，0.1% 的提升通常已经被认为是显著改进。此外，所提模型相对于标准 Transformer 还具有更高效率（见第 4.2.2 节）。这些结果表明，我们的方法能够在保持计算效率的同时，更有效地捕捉长程行为依赖。

Table 1 工业数据集上的方法比较

	Base	SumPooling	TWIN	DIN (Recent50)	DIN	HSTU	Transformer	LONGER
AUC $\uparrow$	0.83968	0.84201	0.84472	0.84698	0.84982	0.84994	0.85111	0.85290
LogLoss $\downarrow$	0.48758	0.48538	0.48168	0.47830	0.47452	0.47490	0.47293	0.47103
Δ AUC(%)	-	+0.28	+0.60	+0.87	+1.21	+1.22	+1.36	+1.57
Δ LogLoss(%)	-	-0.45	-1.21	-1.90	-2.68	-2.60	-3.00	-3.39

4.2.2 消融实验

表 2 给出了对 LONGER 中关键组件以及 query 相关配置的消融实验结果。我们首先考察 TokenMerge 模块与 InnerTrans 组件的影响。与不进行 merge 的基础模型相比，加入 TokenMerge (Concat, 250) 后，FLOPs 从 3.73×10^9 降至 3.03×10^9 ，同时 AUC 提升 1.58%，LogLoss 降低 3.48%。在此基础上进一步引入 InnerTrans 后，模型获得额外增益，取得了最佳 LogLoss 0.47052，以及 1.63% 的 AUC 提升。

接着，我们改变用于概括近期用户行为的 query 数量 k 。结果表明，随着 k 增大，模型性能总体上逐步提升，但计算开销也同步增加。特别地，当采用 100 个 query 时，模型在性能与效率之间取得了较优折中：其 AUC 为 0.85290，LogLoss 为 0.47103，与使用全部 query ($k = 250$) 时的性能非常接近，但 FLOPs 仅为后者的约 54%。表 2 中对这一设置进行了强调，这表明在实际部署中，当计算预算成为关键约束时，该配置具有较高实用性。

最后，我们比较了不同的 query 选择策略。这些策略也可视为 query 集合的不同初始化方式。其中，使用可学习 query（随机初始化）效果最差，AUC 仅为 0.84946。相比之下，直接选取最近的 100 个用户行为（Recent 100）取得了最佳整体性能。其他策略，如均匀采样，或者将最近行为与均匀采样行为混合，虽然也有效，但 AUC 略低、LogLoss 略高。这些发现表明，使用信息量更高的行为来初始化 query，尤其是近期行为，对于长序列建模中有效捕捉用户意图非常关键。

总体而言，该消融实验验证了架构增强（例如 TokenMerge 与 InnerTrans）以及 query 相关设计（例如 query 数量与选择方法）在平衡准确率与效率方面都发挥着关键作用。实验结果说明，通过精心设计关键组件与行为建模流程，LONGER 能够在降低计算成本的同时取得强性能，因此非常适合需要低时延推理和高系统吞吐的大规模工业部署环境。

Table 2 LONGER 中 query 数量与关键组件的消融实验

配置	FLOPs($\times 10^9$)	AUC $\uparrow$	LogLoss $\downarrow$	Δ AUC	Δ LogLoss
LONGER (不含 Merge, 2000)	3.73	0.85111	0.47293	+1.36%	-3.00%
+ TokenMerge4 (Concat, 500)	2.13	0.85232	0.47145	+1.51%	-3.31%
+ TokenMerge8 (Concat, 250)	3.03	0.85291	0.47062	+1.58%	-3.48%
基于 TokenMerge8 再加 InnerTrans	3.52	0.85332	0.47052	+1.63%	-3.50%
改变 query 数量（采样最近的 k 个 item）					
query 数 = 50	1.27	0.85235	0.47162	+1.51%	-3.27%
query 数 = 80	1.59	0.85248	0.47157	+1.52%	-3.28%
query 数 = 100	1.91	0.85290	0.47103	+1.57%	-3.39%
query 数 = 150	2.36	0.85290	0.47101	+1.57%	-3.40%
query 数 = 200	2.93	0.85331	0.47077	+1.62%	-3.45%
query 数 = 250	3.52	0.85332	0.47052	+1.63%	-3.50%
query 选择策略					
Learnable 100	1.91	0.84946	0.47523	+1.17%	-2.53%
Recent 100	1.91	0.85290	0.47103	+1.57%	-3.39%
Uniform 100	1.91	0.85183	0.47215	+1.45%	-3.16%
Recent50 + Rest Unif50	1.91	0.85255	0.47129	+1.53%	-3.34%

4.3 扩展性分析

在这一部分，我们分析模型性能相对于序列长度、FLOPs 以及参数数量的扩展规律。其扩展行为遵循如下通式：

$$y = \alpha x^\beta + \gamma \quad (14)$$

其中， y 表示性能指标（AUC 或 LogLoss）， x 表示扩展因子（序列长度、FLOPs 或参数量）， α 与 β 为常数， γ 表示常数偏置项。

4.3.1 序列长度

我们分析了在不同模型深度下，性能如何随输入序列长度变化而变化。根据原文图 4，随着 token 数量增加，AUC 持续提升、LogLoss 持续下降，并总体呈现幂律趋势。更深的模型能够从更长序列中获得更大收益，但随着深度增加，AUC 增益会逐渐放缓，体现出收益递减现象。因此，最优深度应在模型能力与计算约束之间取得平衡。总体而言，更长序列能够带来更好性能，尤其是在与合适的模型深度搭配时更为明显；但超过一定深度后，额外收益会趋于有限。

4.3.2 参数量

我们通过增大隐藏维度来评估模型容量，同时固定层数为 2、输入序列长度为 2000。如原文图 5(a) 所示，AUC 随参数量增加而稳定提升，并呈现出明显的幂律趋势 ($R^2 = 0.987$)。这些结果表明，在固定架构下，增加模型宽度能够有效提升性能，并且在当前参数范围内尚未观察到明显饱和。

4.3.3 FLOPs

我们在固定模型维度为 32 的前提下，通过改变层数与序列长度来分析模型性能随 FLOPs 的变化情况。如原文图 5(b) 所示，AUC 会随着 FLOPs 增大而稳步提高，并同样呈现显著的幂律趋势 ($R^2 = 0.967$)。这表明，在固定模型宽度下，增加计算资源能够使模型处理更长或更复杂的用户行为序列，从而捕捉更高阶依赖关系并提升预测精度。

这些结果说明，增加计算资源确实是提升性能的有效途径，但其效率收益仍需与真实系统中普遍存在的计算与显存约束进行平衡。

4.4 在线 A/B 测试

在这一部分，我们给出在线 A/B 测试结果，以评估所提模型在真实业务场景中的效果。测试分别在抖音广告平台与抖音电商平台中开展，这两个平台均具有重要商业影响力并覆盖数十亿用户。由于这些场景中的基线模型本身已经非常强，因此所观察到的改进更具意义。跨广告与电商两个领域的双场景测试，也使我们能够评估模型在平台生态中两类关键环境下的泛化能力。

4.4.1 抖音广告平台

这一部分给出抖音广告平台上的 A/B 测试结果。我们使用两个关键指标评估模型表现：ADSS (Advertiser Score) 与 ADVV (Advertiser Value)，它们是工业广告系统中最重要指标。测试覆盖三种广告形态：直播、短视频与商城。

对于直播广告，模型使 ADSS 提升了 1.063%，ADVV 提升了 1.168%。在短视频广告中，ADSS 提升了 2.097%，ADVV 提升了 2.151%。在商城广告中，ADSS 提升了 1.816%，ADVV 提升了 1.407%。这些结果表明，该模型在所有广告形态上都带来了稳定且一致的效果提升。

Table 3 抖音广告 A/B 测试结果

广告类型	ADSS	ADVV
直播	+1.063%	+1.168%
短视频	+2.097%	+2.151%
商城	+1.816%	+1.407%

4.4.2 抖音电商业务

在抖音电商场景的 A/B 测试中，我们使用两个关键指标来评估不同内容形态下的效果：Order/U (每用户订单数) 与 GMV/U (每用户成交总额)。这些指标帮助我们理解模型对总销售规模以及用户层面的参与度与价值两个角度理解模型影响。实验结果显示，这两个指标均获得了显著提升。

在直播内容中，Order/U 提升了 7.9222%，GMV/U 提升了 6.5404%，说明直播内容对于提升用户订单数和用户价值都具有很强正向作用。在短视频内容中，Order/U 提升了 4.6125%，GMV/U 提升了 5.2771%，说明短视频内容同样能够显著提升单用户整体销售表现。总体来看，两种内容形态都带来了可观收益，其中直播在 Order/U 与 GMV/U 两项指标上的提升幅度尤为明显。

Table 4 抖音电商 A/B 测试结果

电商类型	Order / U	GMV / U
直播	+7.9222%	+6.5404%
短视频	+4.6125%	+5.2771%

5 结论

本文提出了 LONGER，这是一种面向工业推荐系统中超长用户行为序列进行高效且可扩展建模的 Transformer 框架。通过引入全局 token、带有 InnerTrans 的 token merge、混合因果注意力，以及包括 GPU 全同步训练框架、混合精度与重计算训练、KV cache serving 在内的系统级优化，LONGER 使得在真实工业约束下进行端到端超长序列建模成为可能。基于工业级十亿规模数据集的线下实验，以及广告与电商两个领域的在线 A/B 测试，共同验证了该方法在十亿级工业用户规模下的鲁棒性与泛化能力。值得注意的是，LONGER 在显著降低计算开销的同时，仍然取得了具有竞争力的精度，因此非常适合部署到对时延敏感的生产环境中。未来工作将包括进一步研究更高效的序列建模技术，并改进工业场景中的跨域行为建模能力。

参考文献说明

原论文参考文献较长，本文为满足“逐段翻译”需求，已完整翻译正文、公式说明、实验表格与结论部分；参考文献条目本身通常以标准英文书目信息形式保留，便于后续检索与引用。